Detecting Malicious Deserialization in the CLR (CVE-2025-53770 like vulnerabilities)

Alexander Rodchenko

Intro

In the summer of 2025, attackers targeted SharePoint servers worldwide, gaining full control over them by using the ToolShell exploit chain. The attacks exploit CVE-2025-49704 and CVE-2025-49706, as well as their patched variants, CVE-2025-53770 and CVE-2025-53771.

The primary **RECOMMENDATIONS** for preventing such intrusions boil down to urgently updating vulnerable SharePoint products and using antivirus solutions capable of detecting ToolShell exploits. However, as experience shows, vulnerable systems often remain unpatched for a long time, and modifications to malware code help attackers evade antivirus detection.

This article describes a general approach to detecting attacks based on deserialization vulnerabilities in the CLR environment, such as CVE-2025-49704 / CVE-2025-53770. At the end of the article, you will find a YARA rule that helps identify suspicious assemblies.

What ToolShell Does

This exploit exists only within the CLR (.NET runtime) and only thanks to deserialization. Deserialization is used to “bring CLR objects back to life” from bytes: to pass them over the network and between processes, restore state from files, databases, logs, and queues, and create snapshots/caches and replicas for fault tolerance and reproducibility. It’s a universal bridge from protocols and storage (REST/gRPC, message brokers, snapshots) to the live object model: object graphs with references and versions are reconstructed without manual glue code, which simplifies integration, migrations, and long-term data storage.

Deserialization is dangerous because, while reading the stream, the runtime can create “unexpected” types and invoke their hidden **CALLBACKS** (e.g., `ISerializable, [OnDeserialized], IDeserializationCallback`) while bypassing constructors - opening the door to gadget chains with side effects up to RCE/DoS.

Particularly vulnerable are mechanisms that embed the type name in the data itself: BinaryFormatter (deemed fundamentally unsafe, marked obsolete, and blocked by default), `NetDataContractSerializer`, `ObjectStateFormatter`. Just look **HERE** by searching for “serial…”

The ToolShell exploit sends a specially crafted POST request to the SharePoint page `/_layouts/15/ToolPane.aspx` with the parameters `MSOtlPn_Uri` and `MSOtlPn_DWP`, which contain the malicious payload. Inside the `MSOtlPn_DWP` parameter, it passes web-part XML markup that SharePoint uses to create an object of type `Microsoft.PerformancePoint.Scorecards.ExcelDataSet` and set its `CompressedDataTable` property to a Base64 string with malicious data.

When attempting to access this object’s `DataTable` property, the provided data is deserialized: the `GetObjectFromCompressedBase64String` method decodes the Base64 string, decompresses it (the data is compressed), and calls the `BinarySerialization.Deserialize` function from the SharePoint library to obtain an object from the byte stream.

The key point I want to highlight is that the vulnerability leads to the execution of managed .NET code that is created dynamically at runtime. In other words, the malicious object produced by deserialization does not initially exist as a loaded DLL—it is formed "on the fly."

The CLR, for its part, can neither execute MSIL directly nor work with the supplied object as-is; before execution, the code is compiled by the JIT compiler into native machine code, and the supplied object is deserialized. The CLR handles all of this on its own (the application developer did not write this code).

Accordingly, at the moment of exploitation we should observe signs of dynamic compilation and code loading within the SharePoint process (IIS/ASP.NET) and, of course, some deserialization activity. This already gives us a starting point for detection (focus on the CLR itself rather than the data). But for now, nothing is clear yet.

What happens when BinaryFormatter.Deserialize is called

In the vulnerable code, the internal `BinarySerialization.Deserialize` is part of ADO.NET; it's the mechanism **HIDDEN** behind `SerializationFormat.Binary`, which internally relies on `BinaryFormatter`, and is therefore also considered unsafe.

When deserialization is invoked with a byte stream, the CLR performs a series of steps to reconstruct objects from that data:

1. **Reading serialization metadata.** The serializer first reads metadata to understand what maps to what. `BinaryFormatter` begins by reading the **SERIALIZATION HEADER**. Naturally, a stream in the .NET Remoting binary format (**NRBF**) contains records with assembly and type names (`BinaryLibrary`, `ClassInfo`, `MemberTypeInfo`, etc.). These are metadata and they're supposed to be there. It's a common set of records that contain strings with fully qualified assembly/type names. Thus, we realize that somewhere there will be a stream where, **IN PLAIN TEXT**, it will say: "**I AM A SUPER-DANGEROUS TYPE, AND I'M ABOUT TO BE DESERIALIZED IN A SUPER-DANGEROUS WAY**."
   Hence a practical idea suggests itself: search memory/files for NRBF streams and match suspicious FQTN (gadget types) with a simple signature scan/YARA based on assembly/type name strings. The format specification and record grammar are officially published by Microsoft, so this is straightforward.
   Therefore, you can enumerate all gadgets and compile a list of suspicious data which, if deserialized, could lead to RCE. And again, this data must reside in the application's memory, and it can be detected with a YARA scanner. That's good.
2. **Loading the assembly and type.** For each object record, the formatter takes the type name plus the assembly's name and attempts to bind them to a `System.Type` in the current loading AppDomain. This behavior is controlled by `BinaryFormatter.Binder` (custom type binding) and `AssemblyFormat`/`FormatterAssemblyStyle` (how to locate the assembly). By **DEFAULT**, it uses normal CLR assembly resolution (via `Assembly.Load` and standard probing paths/contexts); if that fails, `AppDomain.AssemblyResolve` may fire if a handler is registered. In .NET Framework this included the GAC; in .NET (Core/5+) the probing/contexts differ, but the essence is the same: if the assembly/type can be found, they will be loaded/resolved. This is exactly why data-driven polymorphic deserialization is unsafe: the stream dictates which concrete types to instantiate. These loading attempts are clearly visible in ETW under the CLR provider (`Microsoft-Windows-DotNETRuntime`) in the **Loader** category—you can subscribe and see `AssemblyLoad`/`ModuleLoad`.
3. **Object allocation and initialization.** After the type is resolved, the deserializer creates an instance of the object.
   Let's stress this once more: **it is the deserializer that creates the specific *malicious* object instance—and moreover, that object implements a *dangerous* type** (dangerous types are discussed below).
   After resolving the type, the formatter can create the object **without** calling normal constructors and immediately start executing the "magic" deserialization callbacks:

a. **`[Serializable]` without `ISerializable`** → allocation via `FormatterServices.GetUninitializedObject` (in newer versions also `GetSafeUninitializedObject`) plus direct field population; this is dangerous because constructor invariants are bypassed.
b. **`ISerializable` / special ctor `ctor(SerializationInfo, StreamingContext)`** → the constructor runs immediately during deserialization (the class's code is already executing).
c. **`IObjectReference`** → a temporary object is created first, then `GetRealObject()` replaces it with the "real" one (a frequently used step in gadget chains).
d. **`IDeserializationCallback.OnDeserialization` and methods with `[OnDeserialized]`** → invoked after the graph is restored; again, arbitrary type code at a "hot" point.

4. **Graph assembly and fixups.** The CLR maintains an `ObjectManager`: it tracks objects/references/cycles, performs reference fixups, and finally raises deserialization events. Errors at this stage are classic indicators of a corrupted/malicious stream.

`BinaryFormatter` trusts the stream and exposes powerful object life-cycle primitives (allocation without constructors, special constructors, object substitution, callbacks), as well as arbitrary type resolution from available assemblies. This enables gadget chains with side effects up to RCE/DoS. Microsoft officially classifies `BinaryFormatter` as fundamentally unsafe and recommends immediate migration. An excellent high-level corroboration is the official security guidance and the API obsolete/removal status. For a deeper treatment of the technique, see Forshaw's classic write-up (long may he keep creating - my hero).

What kinds of dangerous gadgets exist

An attacker's goal is to inject **gadget objects**: instances of classes whose behavior during deserialization can lead to code execution. Such types are generally not intended for use in business logic (which deals with data, images, and nothing executable), but they are available in the .NET Framework.

They're called **"executable" types** because they contain code-execution points during deserialization (for example, in the serialization constructor or in callbacks).

What happens with a malicious payload?

- **Loading an unexpected type.** The stream may specify a type such as `System.DelegateSerializationHolder`, `System.Data.DataSet`, `System.Workflow.ComponentModel.Serialization.ActivitySurrogateSelector`, and the like. The CLR will load the corresponding assembly (for example, `mscorlib`, `System.Workflow`, `System.Data`) and the type—even if the application never intended to use it.
- **Code execution in the serialization constructor or callbacks.**
  - **Delegates.** If the payload contains `DelegateSerializationHolder+DelegateEntry`, the CLR will create a `DelegateSerializationHolder` and then call `GetRealObject()`. Inside that method, `Delegate.CreateDelegate` is invoked to create a real delegate pointing to the target method - the attacker can specify, for example, `System.Diagnostics.Process.Start`. If the signatures don't match, the CLR will generate a dynamic stub method (adapter), stored in a dynamic assembly, which allows the call to be made with the required signature.
  - **`ActivitySurrogateSelector`.** One of the most powerful gadgets. It is used to **substitute** the serializable type.
  - Using `SerializationInfo.SetType`, the attacker asserts that the object is `System.Windows.Forms.AxHost.State` and adds the key `"PropertyBagBinary"` with an embedded stream
    Inside `AxHost.State`, during deserialization, that data is deserialized **again**—this is the next stage of the attack. The embedded data carries the actual payload: a compiled assembly loaded via `Assembly.Load(byte[])`. When it is loaded, a dynamic assembly with no on-disk PE image appears.
  - Others

- `System.Runtime.Remoting.ObjRef` - can create a proxy to a remote object, even connect to an external server
- `System.Data.DataSet` - can trigger reloading of XML schemas and lead to XML vulnerabilities (XXE, etc.).
- `System.Windows.Data.ObjectDataProvider` - can invoke an arbitrary method during XAML deserialization (in **LosFormatter**; this is the very gadget seen **in the wild** in this case, which we examine here).
- `DesignerVerb`, `ClaimsPrincipal`, `TextFormattingRunProperties` — used in chains where code is invoked, for example, in `ToString()` or within `OnDeserialized`.

All of these classes were originally designed for legitimate purposes—serializing delegates, proxies, and ActiveX state. But they contain code that executes during deserialization, which is precisely what makes them gadgets.
Tools like **YSoSerial.NET** collect dozens of such gadget classes. And it's quite feasible to identify and enumerate these patterns for a scanning **YARA** rule.

What happens on SharePoint

Now to specifics: let's look at what happens on SharePoint itself. First, we'll work extensively with tables and Excel on it, capture a dump, execute the exploit, and then capture another dump.

It was

After exploit:

Assemblies added:

Assem	Addr	Flags
App_Web_toolpane.aspx.9c9699a8.pq2eudb2	0x2523f533a40	
Microsoft.GeneratedCode	0x2523f535780	Dynamic
Microsoft.Web.Design.Server.intl	0x2523f5323c0	
PresentationCore	0x2523f534b20	Native
PresentationFramework	0x2523f532720	Native
System.Data.Services	0x2523f536740	
System.Web.Extensions.Design	0x2523f537820	
Microsoft.Web.Design.Server.intl	0x2523f5323c0	
PresentationCore	0x2523f534b20	Native
PresentationFramework	0x2523f532720	Native

App_Web_toolpane.aspx* — This is the dynamically compiled ASP.NET assembly for the `toolpane.aspx` page. In WebForms, ASP.NET compiles `.aspx` pages *on the* fly into `App_Web_*.dll` and loads them into the application

domain; the mechanism is described in MSDN ("ASP.NET Compilation Overview," "Dynamic Compilation"). This means the corresponding page was indeed requested or explicitly activated by the runtime. In SharePoint, this correlates with the "Tool Pane" utility page, historically hosted at `toolpane.aspx` and used to edit/manage web parts.

Microsoft.GeneratedCode, Dynamic — This is the characteristic name of the dynamic assembly that .NET creates when `XmlSerializer` first serializes/deserializes an unfamiliar type. Generation proceeds via `XmlSerializerFactory` → `XmlSerializer.GenerateTempAssembly(...)` → `TempAssembly`, i.e., a helper assembly is emitted and loaded into the domain. The mechanics are confirmed by the .NET Framework reference sources: the factory always tries either to load a pre-generated `*.XmlSerializers.dll` or to generate a *temp assembly* at runtime. In practice the dynamic assembly name is often `Microsoft.GeneratedCode` (as various reports and field experience confirm), but this is not a strict API contract—rather, de facto behavior of a specific implementation.

And that's exactly what we need: if we find this helper assembly, it means something is being deserialized.

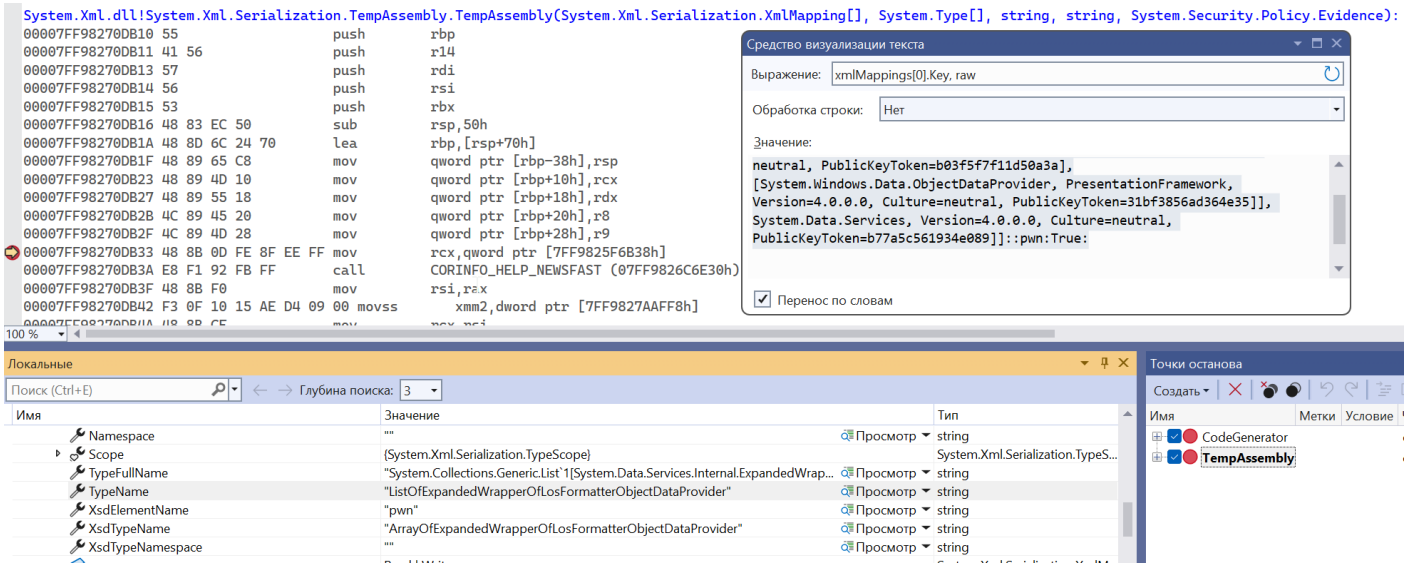
Check the contents of `DumpAfter.txt` (it contains debugger output with commands).

You can see auto-generated fields like `id7_ObjectDataProvider`, `id19_ExpandedWrapperOfLosFormatter`, `id20_LosFormatter` - this directly indicates that the constructed graph included `System.Windows.Data.ObjectDataProvider` (WPF/XAML), `System.Data.Services.Internal.ExpandedWrapper` (WCF Data Services/OData), and `System.Web.UI.LosFormatter` (ASP.NET ViewState). All three are well-known anchor points in deserialization chains.

These APIs are documented in Microsoft Docs: **ObjectDataProvider** (WPF, XAML-based object instantiation), **ExpandedWrapper**<> (internal eager-loading wrappers in ADO.NET Data Services), **LosFormatter** (and it is explicitly classified as unsafe in the documentation).

Let's also attach a debugger and set a breakpoint on `System.Xml.Serialization.TempAssembly.TempAssembly(System.Xml.Serialization.XmlMapping[], System.Type[], string, string, System.Security.Policy.Evidence)`—this is the point where the runtime has already assembled the reflected mappings for the type the serializer is being generated for.

What can we observe there (`debug.txt`; apologies for the formatting of the locals list)?



TypeFullName = "`System.Collections.Generic.List`1[System.Data.Services.Internal.ExpandedWrapper`2[System.Web.UI.LosFormatter,System.Windows.Data.ObjectDataProvider]]`" – Generic List<T> by CLR, where T = `ExpandedWrapper<LosFormatter, ObjectDataProvider>`

Also

- TypeName = "`ListOfExpandedWrapperOfLosFormatterObjectDataProvider`"
- DefaultElementName = "`ArrayOfExpandedWrapperOfLosFormatterObjectDataProvider`"
- Name = "`ObjectDataProvider`"
- Name = "`ProjectedProperty0`"
- Name = "`ExpandedWrapperOfLosFormatterObjectDataProvider`"

This is the work of the internal helpers **`CodeIdentifiers`**, which "flatten" type names (strip the backtick arity from things like "`ExpandedWrapper`2" and glue segments together with "Of").

You can also see:

Key
"`System.Collections.Generic.List`1[[System.Data.Services.Internal.ExpandedWrapper`2[[System.Web.UI.LosFormatter, System.Web, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a],[System.Windows.Data.ObjectDataProvider, PresentationFramework, Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35]], System.Data.Services, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089]]::pwn:True:" - here, the string embeds the assembly-qualified **name (AQN)** of the constructed type, with nested AQNs of the generic arguments, plus the XML element `pwn` (the root name of the malicious parameter via `XmlRoot`).

This is precisely the string that's convenient for signature/YARA search — it contains **absolutely everything**.

However, the data has not yet been deserialized, but an extraordinary number of "suspicious" artifacts have already appeared in the memory of the helper deserialization assembly. In fact, from the single `TypeName`/`Key` string alone, you can see that the serializer must generate code for `List<ExpandedWrapper<LosFormatter,ObjectDataProvider>>`, and below that—inside `ObjectDataProvider`—fields are being serialized that allow invoking an arbitrary method with parameters. This definitely should not occur during "normal" SharePoint operation.

Overall, examining the helper assemblies used for serialization is an excellent way to gather artifacts that indicate the very fact that a payload has arrived which the runtime is about to deserialize into something clearly malicious.

Also, if you monitor what gets invoked within the `CodeGenerator` namespaces, you will notice the method builder (in our case, four methods) that will construct the methods, and you can try to capture `m_ILStream` once the dynamic method is ready. I'm not including the debugger listing here, but I did add this artifact to the rule—it will be repeatable (this is not an NGEN method, so it will be identical when constructing methods responsible for serializing the same types). I took a substantial portion of the IL stream that appeared for data alignment. Empirically, this catches malicious deserializers and does not show up in benign ones. That said, it's somewhat redundant.

These checks are exactly what we'll use in the YARA rule that locates the helper assembly being used for malicious deserialization.

Overall detection workflow

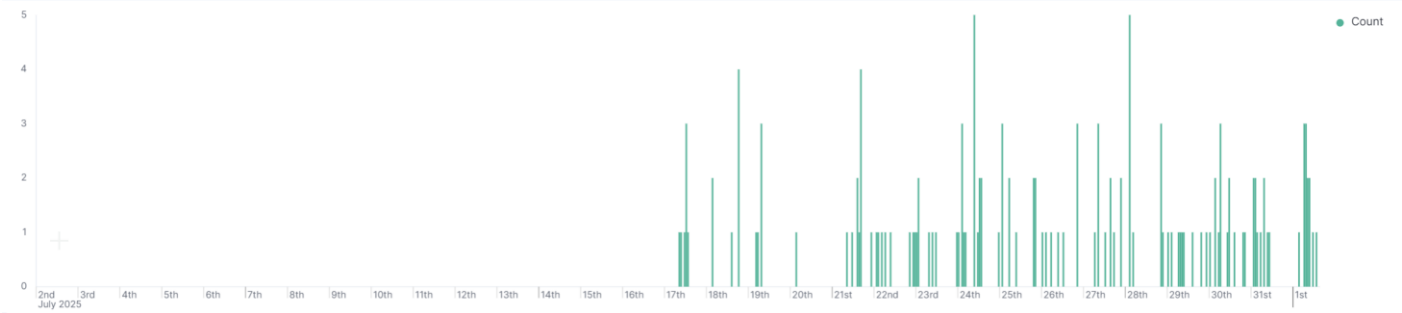
Now let's describe, end-to-end, how a deserialization-exploit detection system can work:

1. The EDR solution's ETW monitor starts with the system (or listens continuously). It subscribes to CLR **Loader** events.
2. Upon receiving a Loader event (logical AND):
    a. **Dynamic** `Microsoft.GeneratedCode` with **no path**
    b. `App_Web_toolpane.aspx*` with **no path**

3. Run the YARA rule, as shown above, to verify that `Microsoft.GeneratedCode` is the helper that is deserializing a gadget for the vulnerable parameter. If that's confirmed, raise a high-severity alert—and you may even perform host forensics.
4. Scan the memory of the same application domain for patterns of known gadgets. If found, that is a **100%** exploitation.

And there will be PoC + Demo that detects an exploit attempt (code nonpublic now)

At the time of writing, our MDR monitoring logged **187,132** events over the last 30 days where IIS (which hosts just about anything) loaded `Microsoft.GeneratedCode`. Yes—IIS services perform deserialization. However, here is what the 30-day statistics look like for loading the compiled ASP assembly `App_Web_toolpane` into the application domain where SharePoint runs:



There are few such events—which makes this a good candidate for an alert. And by locating the "right" helper and scanning for the presence of dangerous gadgets, you can virtually eliminate false positives.
Here is an example YARA rule (I took the strings from `DumpAfter.txt`, the data from `m_ILStream`, and the debugger):

```
rule ToolShell_MicrosoftGeneratedCode_XmlGadget
{
    meta:
        description = "XmlSerializer helper assembly loaded during ToolShell exploit. Based on
pwn object deserialization"
        author = "Rodchenko"
        reference = "CVE-2025-53770 / ExpandedWrapper + LosFormatter chain"
        hash_mode = "memory "

    strings:
        /* IL-signatures, from m_ILStream when
           Dynamic method "Microsoft.GeneratedCode" was on its building state
           https://stackoverflow.com/a/4147132/6300544
        */
        $il4 = { 17 3B 00 00 00 00 05 28 0C 00 00 0A 0A 06 D0 0A 00 00 01 28 0D 00 00 0A FE 01
39 00 00 00 00 38 00 00 00 00 02 05 28 0E 00 00 0A 7A 02 03 04 05 16 14 28 0F 00 00 0A 0E 05 }

        $data =
"System.Collections.Generic.List`1[[System.Data.Services.Internal.ExpandedWrapper`2[[System.Web
.UI.LosFormatter, System.Web, Version=4.0.0.0, Culture=neutral,
PublicKeyToken=b03f5f7f11d50a3a],[System.Windows.Data.ObjectDataProvider,
PresentationFramework, Version=4.0.0.0, Culture=neutral, PublicKeyToken=31bf3856ad364e35]],
System.Data.Services, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b77a5c561934e089]]"
        $TypeName = "ListOfExpandedWrapperOfLosFormatterObjectDataProvider"

        $idpwn = "id1_pwn"
        /*
        $id2 = "id2_Item"
        $id3 = "id3_Item"
        $id4 = "id4_Description"
        $id5 = "id5_ExpandedElement"
        */
        $id6 = "id6_ProjectedProperty0"
```

```
        $id7 = "id7_ObjectDataProvider"
        /*
        $id8 = "id8_IsInitialLoadEnabled"
        $id9 = "id9_ObjectType"
        $id10 = "id10_ObjectInstance"
        $id11 = "id11_MethodName"
        $id12 = "id12_ConstructorParameters"
        */
        $id13 = "id13_anyType"
        /*
        $id14 = "id14_MethodParameters"
        $id15 = "id15_IsAsynchronous"
        $id16 = "id16_MemberInfo"
        $id17 = "id17_Type"
        $id18 = "id18_DataSourceProvider"
        */
        $id19 = "id19_ExpandedWrapperOfLosFormatter"
        $id20 = "id20_LosFormatter"
        /*
        $id21 = "id21_Item"
        */
    condition:
        $il4 or $data or $TypeName or $id19 or $id7 or $id6 or ($idpwn and 3 of ($id*))
}
```